Announcements 📣

- Phase 3 of the project is due this week Saturday at 11:59 PM
- Quiz 2 (React) on Monday, **October 26th**

Web Engineering & Development (SWE 363)

Interactive Front-End Development

In today's lecture:

- Conditional Rendering
- Lists (React)

Reference:

- Zybook: 5.10-5.11

5.10 Conditional Rendering

What is Conditional Rendering?

Conditional rendering means showing or hiding parts of the UI depending on certain conditions.

Real-world examples:

- Show "Login" button when user is logged out
- Display "Loading..." while data is being fetched
- Show "No tasks yet" when a list is empty
- Hide error messages when there are no errors

Why is Conditional Rendering Useful?

- **Dynamic content** that reacts to data changes
- **Better user experience** by showing relevant information
- **Cleaner UI** by avoiding unnecessary clutter

Three Ways to Do Conditional Rendering

1. if Statements

```
function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  } else {
    return <h1>Please log in.</h1>;
  }
}
```

Three Ways to Do Conditional Rendering

2. Ternary Operator

```
{isLoading ? <Spinner /> : <DataList />}
```

3. Logical AND

```
{error && <div className="error">{error}</div>}
```

Conditional Rendering: if Statements

When to use:

- **Complex logic** that's hard to read inline
- **Multiple conditions** to check
- **Different components** to return entirely

Conditional Rendering: Ternary Operator

When to use:

- **Simple true/false** conditions
- **Two different elements** to show
- **Inline rendering** within JSX

Syntax:

```
condition ? trueValue : falseValue
```

Conditional Rendering: Logical AND

When to use:

- **Show/hide** a single element
- **Simple conditions** that are either true or false
- **Optional content** that only appears when needed

Syntax:

```
condition && <Element />
```

Common Conditional Rendering Patterns

Pattern 1: Empty State

```
if(tasks.length === 0) {
  return <p>No tasks yet. Add your first one!</p>
} else {
  return <ul>
    {tasks.map(task => <TaskItem key={task.id} task={task} />)}
  </ul>
}
```

Common Conditional Rendering Patterns

Pattern 2: Loading State

```
{isLoading ?
  return <div>Loading...</div>
} else {
  return <DataComponent data={data} />
}
```

Pattern 3: Error Handling

```
{error && <ErrorMessage error={error} />}
{!error && <MainContent />}
```

5.11 Lists

Lists in React - What are Lists in React?

Lists are collections of similar items displayed dynamically.

Real-world examples:

- Shopping cart items
- User comments on a post
- Todo tasks
- Product catalog
- Search results

Lists in React - How to render Lists in React?

Use `.map()`

- **Dynamic rendering** based on data
- **Automatic updates** when data changes
- **Reusable components** for each item
- **Clean, declarative code**

Lists in React - How to render Lists in React?

Basic Syntax:

```
{items.map(item => (
  <Component key={item.id} data={item} />
))}
```

Step-by-step:

1. **Start with an array** of data
2. **Use** `.map()` to transform each item
3. **Return JSX** for each item
4. **Add a unique** `key` prop

The Importance of Keys

What are keys?

Keys are unique identifiers that help React track which items have changed, been added, or removed.

Why are they important?

- **Performance**: React can efficiently update only changed items
- **State preservation**: Form inputs keep their values
- **Animation**: Smooth transitions between states
- **Prevents bugs**: Avoids UI glitches

Key Best Practices

Good Keys:

```
// Use unique, stable IDs
{users.map(user => (
  <UserCard key={user.id} user={user} />
))}

// Use unique properties
{products.map(product => (
  <ProductCard key={product.sku} product={product} />
))}
```

Key Best Practices

Bad Keys:

```
// Don't use array index for dynamic lists
{tasks.map((task, index) => (
  <TaskItem key={index} task={task} />  // X Bad!
))}

// Don't use random values
{tasks.map(task => (
  <TaskItem key={Math.random()} task={task} />  // X Bad!
))}
```

When Index is OK for Keys

Safe to use index when:

- **List never reorders** (items stay in same position)
- **Items are never added/removed** from middle
- **Simple, static lists**

Example:

```
// Static menu items - order never changes
const menuItems = ['Home', 'About', 'Contact'];

{menuItems.map((item, index) => (
  <MenuItem key={index} text={item} />
))}
```

30m

Demo

5.4 More React

- Command to get process access in the lab PCs

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Next Class

- More on React